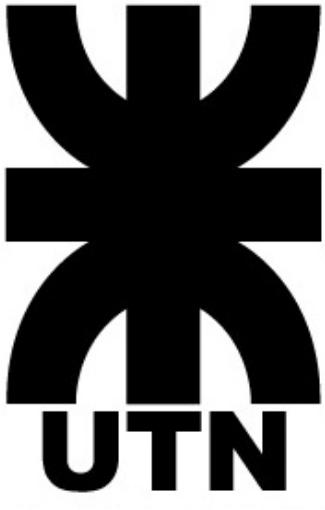
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL MENDOZA



Ingeniería y Calidad de Software
Trabajo Práctico N°6

GRUPO Nº4 - 4K10
MÓDULO 1 Y 4

Integrantes:

- Barchiesi Santino 50791
- Estudillo Máximo 50013
- Navarro Matías 51006

Repositorio y organización del equipo 3
Responsabilidades de cada integrante 3
Tarea A: Frontend mínimo 4
Descripción 4
Flujos implementados 4
Criterios de aceptación 7
Tarea B: Tests E2E con Cypress 8
Organización 8
Código de los tests 8
Ejecución y evidencia 13
Análisis de resultados 18
Tarea C: Tests unitarios con IA 20
Organización 20
Uso de IA 20
Tests y uso de IA por integrante 21
Evidencia de ejecución 29
Reflexión estructurada 32
Sobre la trazabilidad 32
Sobre el valor de testear 32
Sobre el uso de IA 32
Lecciones aprendidas 33

Repositorio y organización del equipo

Repositorio GitHub del equipo: https://github.com/maxidoe2/agendaya-tp6-grupo4

En el repositorio están el frontend, el servidor, los tests E2E y unitarios, las capturas de las ejecuciones y este informe. El README explica cómo levantar la aplicación y cómo correr cada tipo de test. Hasta la entrega se hicieron 9 commits, en este orden: base del TP5, extracción de la lógica de negocio, frontend con data-cy, tests E2E, tests unitarios y documentación.

Como base se tomó la aplicación del TP5 (https://github.com/maxidoe2/agendaya-tp5-grupo4).

Responsabilidades de cada integrante

Cada integrante tiene al menos un test E2E y cinco tests unitarios sobre dos o más funciones.

Integrante	Tests E2E (cypress/e2e)	Tests unitarios (tests/unit)
Barchiesi Santino	m01-registro-exitoso (1) m01-login-logout (2)	m01-email (8) m01-seguridad (8)
Estudillo Máximo	m01-registro-errores (2) m04-reserva-errores (3)	m01-password-registro (8) m04-validacion-conflictos (10)
Navarro Matías	m04-reserva-exitosa (2)	m04-slots-antelacion (11)
Total	10	45

El frontend lo armamos entre los tres y todos conocemos cómo está organizado.

Tarea A: Frontend mínimo

Descripción

Partimos de la aplicación del TP5: un servidor en Node.js con Express, las pantallas en HTML, CSS y JavaScript, los datos en un archivo JSON y los correos registrados en data/correos.log. El enunciado permite no tener backend, pero lo mantuvimos porque ya estaba hecho y nos deja comprobar en los tests que la reserva realmente se guardó y que los correos se dispararon.

Para que la aplicación se pudiera testear hicimos estos cambios:

- Sacamos las reglas de negocio de server.js y de los scripts de cada pantalla y las pasamos a src/logica/validaciones.js (M01) y src/logica/reservas.js (M04). Son funciones que no dependen de la pantalla ni del servidor, así que se pueden probar con Jest. El mismo archivo lo usa el navegador para validar el formulario y el servidor para validar el pedido.
- En el TP5 los horarios estaban fijos al 15/09/2026, que ya pasó. Ahora el invitado elige un día y la agenda se arma según la disponibilidad: lunes a viernes de 9 a 17 h, turnos de 60 minutos y 2 horas de anticipación mínima. La disponibilidad le corresponde al módulo M02, así que la dejamos fija.
- Las funciones que dependen de la hora actual reciben la fecha como parámetro (ahora), para que los tests den siempre el mismo resultado sin importar el día en que se corren.
- Agregamos el atributo data-cy a todos los elementos interactivos y a los mensajes que se verifican en los tests, por ejemplo email-input, login-button, submit-booking, slot-1000, alerta-error y booking-confirmation.

Flujos implementados

Módulo	Flujo	Pantalla	Requerimiento
M01	Registro de usuario	registro.html	AYA-M01-RF-01
M01	Login y logout	index.html → dashboard.html → Cerrar sesión	AYA-M01-RF-04
M04	Seleccionar fecha y hora	reservar.html	AYA-M04-RF-03
M04	Completar formulario y confirmar reserva	reservar.html	AYA-M04-RF-03 y RF-05
M04 (opcional)	Manejo de horario no disponible	reservar.html	AYA-M04-RF-05

También quedaron de la entrega anterior la recuperación y el restablecimiento de la contraseña, con sus data-cy.

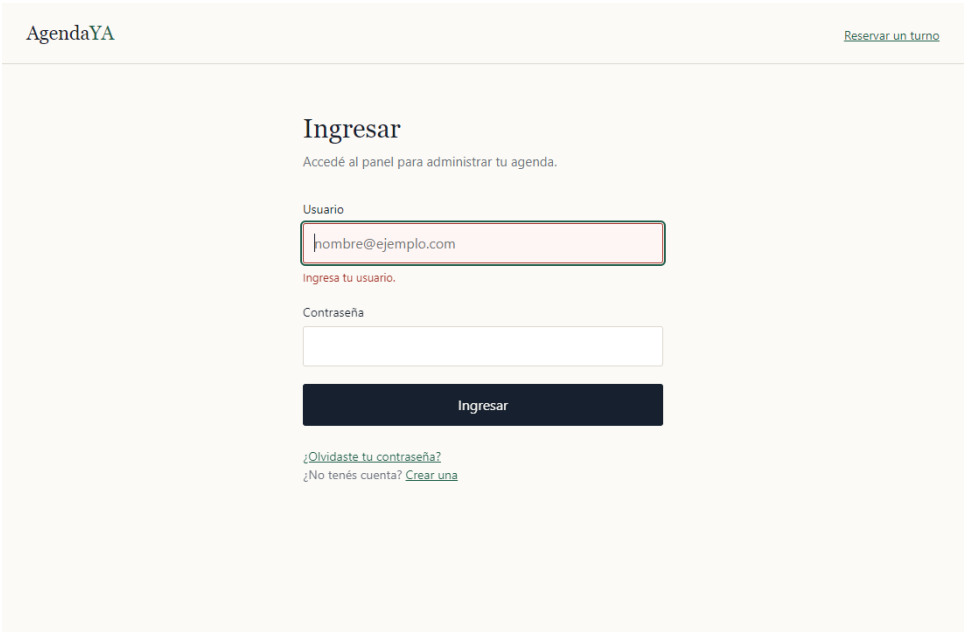


Figura 1 – Login con los campos vacíos: se muestra el error debajo del campo.

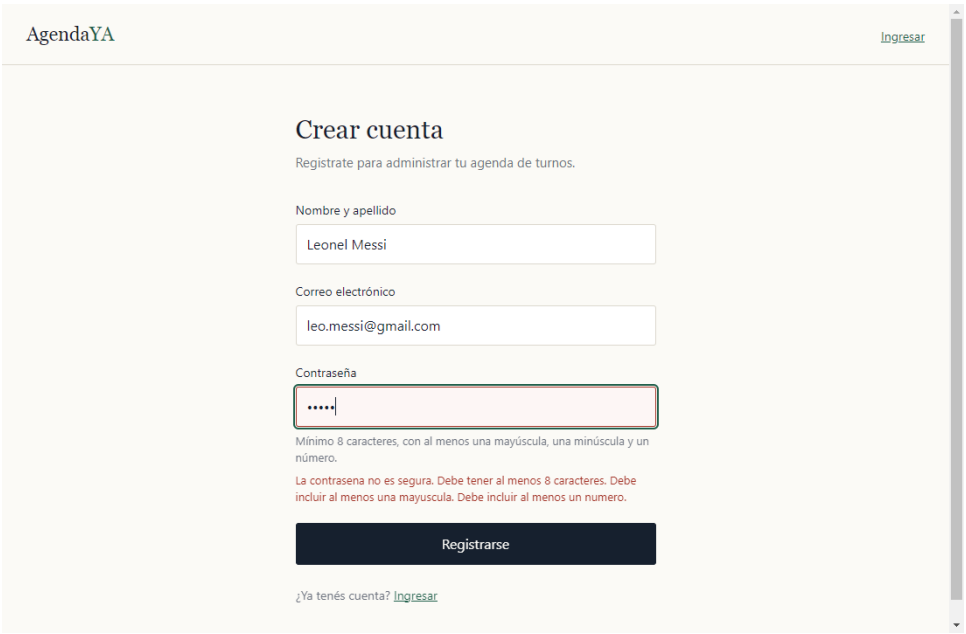


Figura 2 – Registro con una contraseña que no cumple los requisitos.

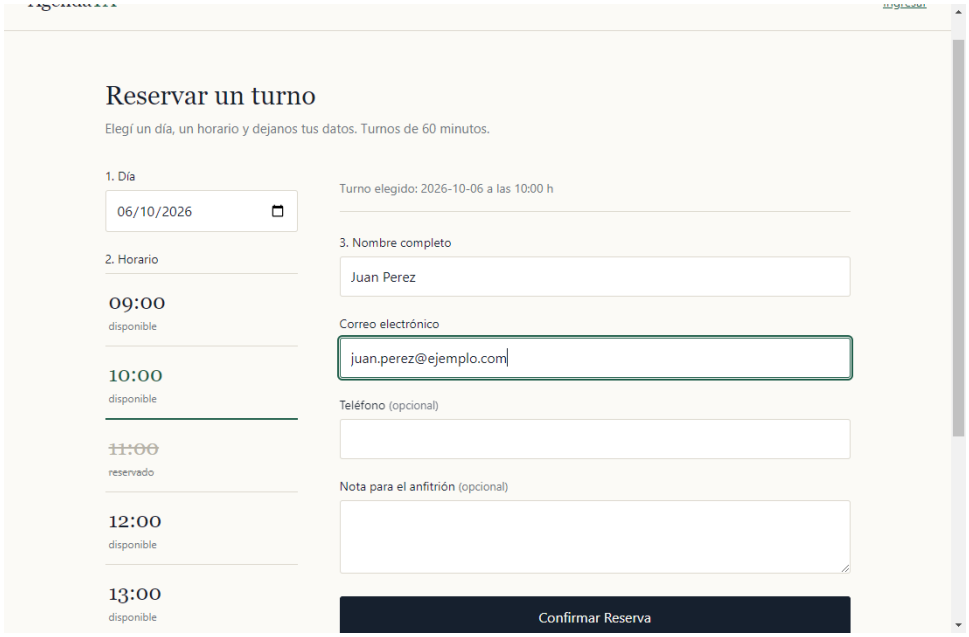


Figura 3 – Reserva: día y horario elegidos. El turno de las 11:00 ya está reservado.

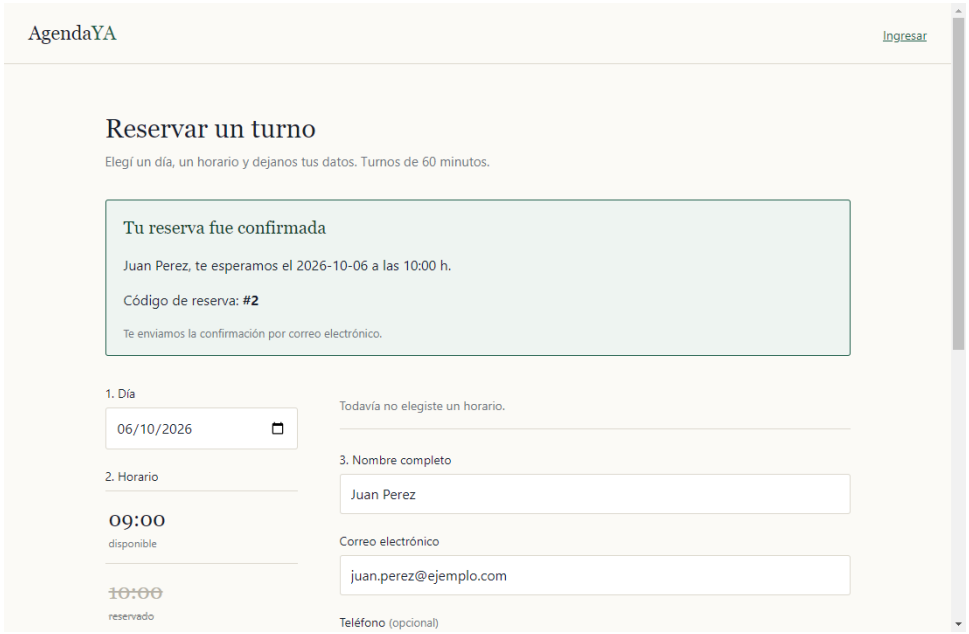


Figura 4 – Reserva confirmada con su código.

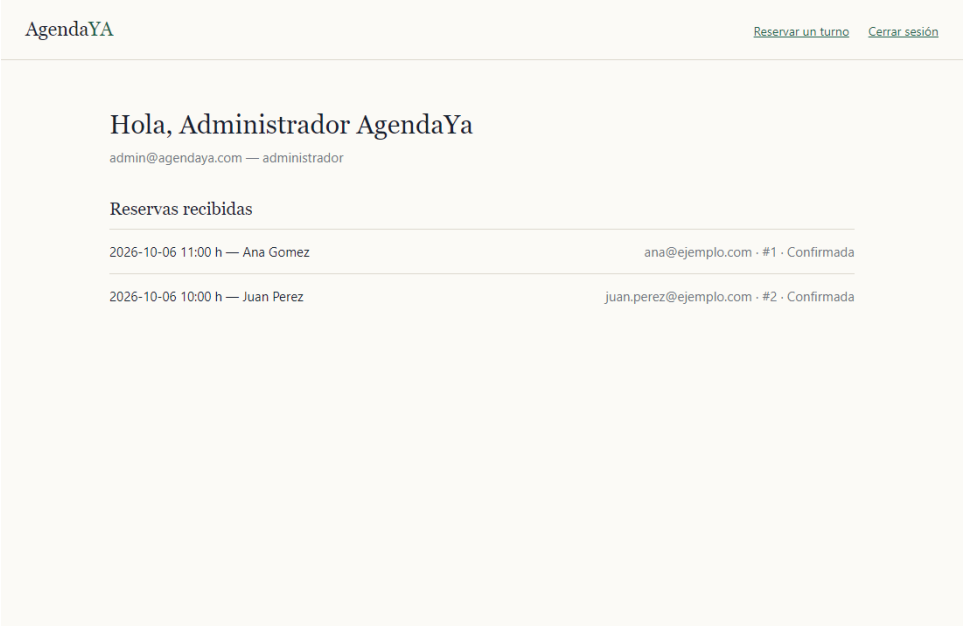


Figura 5 – Panel del administrador con las reservas recibidas.

Criterios de aceptación

Criterio	Cumplimiento
Los dos flujos obligatorios se pueden recorrer sin errores	Sí. Los recorren de punta a punta los tests E2E, que pasan los 10.
Todos los elementos interactivos tienen data-cy	Sí. Buscamos en todas las pantallas inputs, botones, enlaces y formularios sin el atributo y no quedó ninguno. Los botones de los turnos se crean desde JavaScript con data-cy="slot-HHMM".
Los formularios validan el campo vacío y muestran un error visible	Sí. Login, registro, recuperación y reserva muestran el mensaje debajo del campo. Lo verifican m01-registro-errores y m04-reserva-errores.
Al terminar un flujo se muestra una confirmación	Sí. Registro: "Tu cuenta fue creada correctamente.". Logout: "Cerraste sesión correctamente.". Reserva: recuadro con el código de la reserva.
Repositorio Git con al menos 3 commits descriptivos	Sí, 9 commits.

Tarea B: Tests E2E con Cypress

Organización

- La URL base (http://localhost:3000) está una sola vez en cypress.config.js y los tests usan rutas relativas.
- Antes de cada test se reinicia la base de datos con cy.task("reiniciarBase"), así cada test empieza igual y no depende de los demás.
- Además de la pantalla, revisamos lo que no se ve: los correos enviados (cy.task("leerCorreos")), las reservas guardadas (cy.request("/api/reservas")) y los pedidos al servidor con su código de respuesta (cy.intercept).
- Los tests de reserva usan el próximo día hábil. Con una fecha fija, el test dejaría de andar cuando esa fecha pase.
- Al final de cada test se guarda una captura de Cypress con la pantalla y la lista de comandos.

Integrante	Archivo	Tests	Tipo
Barchiesi Santino	m01-registro-exitoso	Registro de un administrador nuevo	Caso exitoso
Barchiesi Santino	m01-login-logout	Login, logout y panel protegido Contraseña incorrecta	Caso exitoso Datos inválidos
Estudillo Máximo	m01-registro-errores	Formulario vacío Correo ya registrado	Datos inválidos Estado del sistema
Estudillo Máximo	m04-reserva-errores	Horario tomado por otro invitado Falta el correo Día no hábil	Estado del sistema Datos inválidos Caso borde
Navarro Matías	m04-reserva-exitosa	Fecha, hora y confirmación Varios clicks seguidos	Caso exitoso Estado del sistema

Código de los tests

Barchiesi Santino: Registro exitoso

Archivo: cypress/e2e/m01-registro-exitoso.cy.js

```
/**
 * M01 - Autenticacion | Flujo obligatorio: Registro de usuario (happy path)
```

```
 * Responsable: Barchiesi Santino
 * Trazabilidad: AYA-M01-RF-01 / CP-003
 */
describe("AgendaYA - M01 Autenticacion: registro exitoso", () => {
  beforeEach(() => {
    cy.visit("/registro.html");
  });

  it("registra un administrador nuevo y lo deja dentro del panel", () => {
    // Arrange: completar el formulario con datos validos y un correo nuevo
    cy.get('[data-cy="nombre-input"]').type("Leonel Messi");
    cy.get('[data-cy="email-input"]').type("leo.messi@gmail.com");
    cy.get('[data-cy="password-input"]').type("Messimessi12");

    // Act: enviar el registro
    cy.get('[data-cy="registro-button"]').click();

    // Assert: se crea la cuenta y se accede al panel con la sesion del usuario nuevo
    cy.url().should("include", "/dashboard");
    cy.get('[data-cy="alerta-exito"]')
      .should("be.visible")
      .and("contain", "Tu cuenta fue creada correctamente.");
    cy.get('[data-cy="dashboard-welcome"]').should("have.text", "Hola, Leonel Messi");
    cy.get('[data-cy="usuario-email"]').should("contain", "leo.messi@gmail.com");
  });
});
```

Barchiesi Santino: Login y logout

Archivo: cypress/e2e/m01-login-logout.cy.js

```
/**
 * M01 - Autenticacion | Flujo obligatorio: Login y logout
 * Responsable: Barchiesi Santino
 * Trazabilidad: AYA-M01-RF-04 / CP-001 y CP-002
 */
describe("AgendaYA - M01 Autenticacion: login y logout", () => {
  beforeEach(() => {
    cy.visit("/");
  });

  it("inicia sesion, entra al panel y al cerrar sesion pierde el acceso", () => {
    // Arrange: credenciales del administrador precargado
    cy.get('[data-cy="email-input"]').type("admin@agendaya.com");
    cy.get('[data-cy="password-input"]').type("Contrasena123!");

    // Act (1): iniciar sesion
    cy.get('[data-cy="login-button"]').click();

    // Assert (1): llega al panel con su usuario
    cy.url().should("include", "/dashboard");
    cy.get('[data-cy="dashboard-welcome"]').should("have.text", "Hola, Administrador
AgendaYa");

    // Act (2): cerrar sesion
    cy.get('[data-cy="logout-button"]').click();

    // Assert (2): vuelve al login con confirmacion y el panel queda protegido
    cy.location("pathname").should("eq", "/");
    cy.get('[data-cy="alerta-exito"]').should("be.visible").and("have.text", "Cerraste sesión
correctamente.");
    cy.visit("/dashboard.html");
    cy.location("pathname").should("eq", "/");
```

```
  });

  it("con contrasena incorrecta no ingresa y limpia el campo de contrasena", () => {
    // Arrange: usuario existente con una contrasena equivocada
    cy.get('[data-cy="email-input"]').type("admin@agendaya.com");
    cy.get('[data-cy="password-input"]').type("ContrasenaInvalida123");

    // Act: intentar ingresar
    cy.get('[data-cy="login-button"]').click();

    // Assert: mensaje exacto de CP-002, campo vacio y sin acceso al panel
    cy.get('[data-cy="alerta-error"]')
      .should("be.visible")
      .and("have.text", "Contrasena incorrecta, intentelo de nuevo.");
    cy.get('[data-cy="password-input"]').should("have.value", "");
    cy.location("pathname").should("eq", "/");
  });
});
```

Estudillo Máximo: Registro con errores

Archivo: cypress/e2e/m01-registro-errores.cy.js

```
/**
 * M01 - Autenticacion | Flujo obligatorio: Registro de usuario (variantes de error)
 * Responsable: Estudillo Maximo
 * Trazabilidad: AYA-M01-RF-01 / CP-004
 */
describe("AgendaYA - M01 Autenticacion: registro con errores", () => {
  beforeEach(() => {
    cy.intercept("POST", "/api/registro").as("registro");
    cy.visit("/registro.html");
  });

  it("no envia el formulario vacio y explica que falta el nombre", () => {
    // Arrange: formulario sin completar (estado inicial de la pantalla)
    cy.get('[data-cy="nombre-input"]').should("have.value", "");

    // Act: intentar registrarse sin datos
    cy.get('[data-cy="registro-button"]').click();

    // Assert: error visible en el campo, sin pedido al servidor y sin salir de la pantalla
    cy.get('[data-cy="error-nombre"]').should("be.visible").and("have.text", "Ingresa tu
nombre.");
    cy.get("@registro.all").should("have.length", 0);
    cy.url().should("include", "/registro.html");
  });

  it("rechaza un correo que ya esta registrado y no crea la cuenta", () => {
    // Arrange: ya existe una cuenta con ese correo (precondicion de CP-004)
    cy.request("POST", "/api/registro", {
      nombre: "Leonel Messi",
      email: "leo.messi@gmail.com",
      password: "Messimessi12",
    });
    cy.get('[data-cy="nombre-input"]').type("Leonel Messi");
    cy.get('[data-cy="email-input"]').type("leo.messi@gmail.com");
    cy.get('[data-cy="password-input"]').type("Messimessi13");

    // Act: intentar registrar el mismo correo otra vez
    cy.get('[data-cy="registro-button"]').click();
```

```
    // Assert: el servidor responde 409, se informa el motivo y no se entra al panel
    cy.wait("@registro").its("response.statusCode").should("eq", 409);
    cy.get('[data-cy="alerta-error"]')
      .should("be.visible")
      .and("have.text", "El correo ya existe en el sistema.");
    cy.get('[data-cy="error-email"]').should("be.visible");
    cy.url().should("include", "/registro.html");
  });
});
```

Estudillo Máximo: Reserva con errores

Archivo: cypress/e2e/m04-reserva-errores.cy.js

```
/**
 * M04 - Booking publico | Variantes de error del flujo de reserva:
 *   - por estado del sistema: horario no disponible (flujo opcional del modulo)
 *   - por datos invalidos: falta el correo electronico
 * Responsable: Estudillo Maximo
 * Trazabilidad: AYA-M04-RF-03 y AYA-M04-RF-05 / CP-008 / enunciado 3.1
 */
import { proximoDiaHabil, proximoSabado } from "../support/fechas";

describe("AgendaYA - M04 Booking: reserva con errores", () => {
  const dia = proximoDiaHabil();

  beforeEach(() => {
    cy.intercept("POST", "/api/reservas").as("crearReserva");
    cy.visit("/reservar.html");
  });

  it("informa que el horario ya fue reservado si otro invitado lo tomo antes", () => {
    // Arrange: el invitado elige las 15:00 y completa sus datos...
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1500"]').should("be.enabled").click();
    cy.get('[data-cy="nombre-input"]').type("Ana Gomez");
    cy.get('[data-cy="email-input"]').type("ana.gomez@ejemplo.com");
    // ...mientras tanto otro invitado reserva ese mismo turno desde otro dispositivo
    cy.request("POST", "/api/reservas", {
      nombre: "Otro Invitado",
      email: "otro@ejemplo.com",
      fecha: dia,
      hora: "15:00",
    });

    // Act: confirmar la reserva sobre un turno que ya no esta libre
    cy.get('[data-cy="submit-booking"]').click();

    // Assert: conflicto 409 con el mensaje del enunciado, agenda actualizada y
    // el turno sigue siendo del primer invitado
    cy.wait("@crearReserva").its("response.statusCode").should("eq", 409);
    cy.get('[data-cy="alerta-error"]')
      .should("be.visible")
      .and("have.text", "Este horario ya fue reservado. Por favor elegí otro.");
    cy.get('[data-cy="slot-1500"]').should("be.disabled").and("contain", "reservado");
    cy.get('[data-cy="booking-confirmation"]').should("not.be.visible");
    cy.get('[data-cy="submit-booking"]').should("be.enabled");
    cy.request("/api/reservas").its("body").should("have.length", 1)
      .its(0).its("email").should("eq", "otro@ejemplo.com");
  });

  it("no envia la reserva si falta el correo y resalta el campo", () => {
    // Arrange: horario elegido y solo el nombre completo (CP-008)
```

```
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-0900"]').click();
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");

    // Act: intentar confirmar
    cy.get('[data-cy="submit-booking"]').click();

    // Assert: mensaje visible en el campo y ningun pedido al servidor
    cy.get('[data-cy="error-email"]')
      .should("be.visible")
      .and("have.text", "Ingresa tu correo electronico para confirmar la reserva.");
    cy.get("@crearReserva.all").should("have.length", 0);
    cy.get('[data-cy="booking-confirmation"]').should("not.be.visible");
  });

  it("un sabado no ofrece horarios", () => {
    // Arrange: dia no habil segun la disponibilidad del administrador (M02)
    const sabado = proximoSabado();

    // Act: elegir el sabado en el calendario
    cy.get('[data-cy="fecha-input"]').type(sabado);

    // Assert: no se muestra ningun turno y se explica por que
    cy.get('[data-cy="agenda-vacia"]')
      .should("be.visible")
      .and("have.text", "Ese dia no hay turnos disponibles. Elegi otro dia.");
    cy.get('[data-cy^="slot-"]').should("not.exist");
  });
});
```

Navarro Matías: Reserva exitosa

Archivo: cypress/e2e/m04-reserva-exitosa.cy.js

```
/**
 * M04 - Booking publico | Flujos obligatorios: Seleccionar fecha y hora +
 * Completar formulario y confirmar reserva (happy path)
 * Responsable: Navarro Matias
 * Trazabilidad: AYA-M04-RF-03 y AYA-M04-RF-05 / CP-007 y CP-010
 */
import { proximoDiaHabil } from "../support/fechas";

describe("AgendaYA - M04 Booking: reserva exitosa", () => {
  const dia = proximoDiaHabil();

  beforeEach(() => {
    cy.visit("/reservar.html");
  });

  it("elige dia y horario, confirma la reserva y notifica a ambas partes", () => {
    // Arrange: elegir un dia habil futuro, un horario libre y completar los datos
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1000"]').should("be.enabled").click();
    cy.get('[data-cy="resumen-turno"]').should("contain", dia + " a las 10:00 h");
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");
    cy.get('[data-cy="email-input"]').type("juan.perez@ejemplo.com");

    // Act: confirmar la reserva
    cy.get('[data-cy="submit-booking"]').click();

    // Assert: confirmacion visible, turno ocupado en la agenda y correos disparados
    cy.get('[data-cy="booking-confirmation"]')
```

```
      .should("be.visible")
      .and("contain", "Tu reserva fue confirmada")
      .and("contain", dia);
    cy.get('[data-cy="booking-codigo"]').should("have.text", "#1");
    cy.get('[data-cy="submit-booking"]').should("be.disabled");
    cy.get('[data-cy="slot-1000"]').should("be.disabled").and("contain", "reservado");
    cy.task("leerCorreos").then((correos) => {
      expect(correos).to.contain("Para:   juan.perez@ejemplo.com");
      expect(correos).to.contain("Para:   admin@agendaya.com");
    });
  });

  it("ante varios clicks rapidos en Confirmar registra una sola reserva", () => {
    // Arrange: formulario listo para confirmar
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1100"]').click();
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");
    cy.get('[data-cy="email-input"]').type("juan.perez@ejemplo.com");

    // Act: tres clicks seguidos sobre el boton (force: el boton ya se deshabilita)
    cy.get('[data-cy="submit-booking"]').click().click({ force: true }).click({ force: true
});

    // Assert: la reserva se confirma y en el sistema hay un unico registro
    cy.get('[data-cy="booking-confirmation"]').should("be.visible");
    cy.request("/api/reservas").its("body").should("have.length", 1);
  });
});
```

Ejecución y evidencia

Los tests se ejecutaron con npm run test:e2e, que levanta el servidor y corre npx cypress run:

```
  (Run Finished)


      Spec                                              Tests  Passing  Failing  Pending
Skipped
  +-------------------------------------------------------------------------------------------
-----+
  | OK  m01-login-logout.cy.js                  00:06        2        2        -        -
- |
  +-------------------------------------------------------------------------------------------
-----+
  | OK  m01-registro-errores.cy.js              00:04        2        2        -        -
- |
  +-------------------------------------------------------------------------------------------
-----+
  | OK  m01-registro-exitoso.cy.js              00:03        1        1        -        -
- |
  +-------------------------------------------------------------------------------------------
-----+
  | OK  m04-reserva-errores.cy.js               00:08        3        3        -        -
- |
  +-------------------------------------------------------------------------------------------
-----+
  | OK  m04-reserva-exitosa.cy.js               00:07        2        2        -        -
- |
  +-------------------------------------------------------------------------------------------
-----+
```

```
    OK  All specs passed!                    00:30       10       10        -        -
-
```

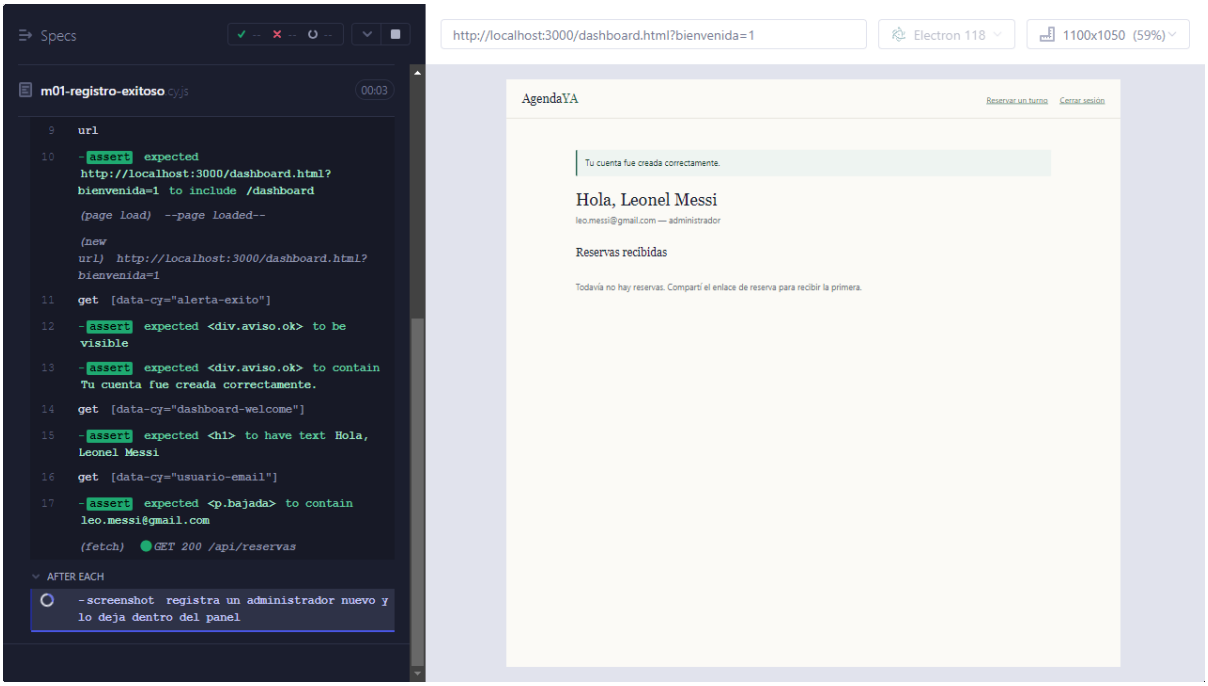


Figura 6 – m01-registro-exitoso (Santino): registro y acceso al panel.

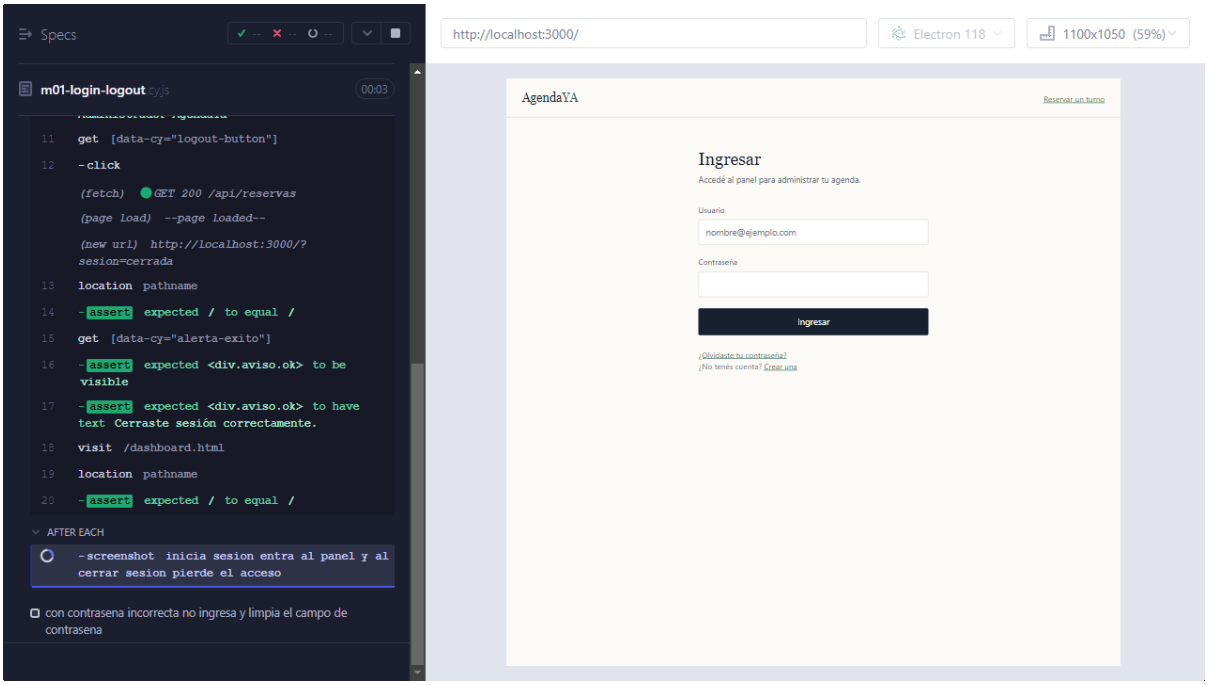


Figura 7 – m01-login-logout (Santino): después del logout el panel vuelve al login.

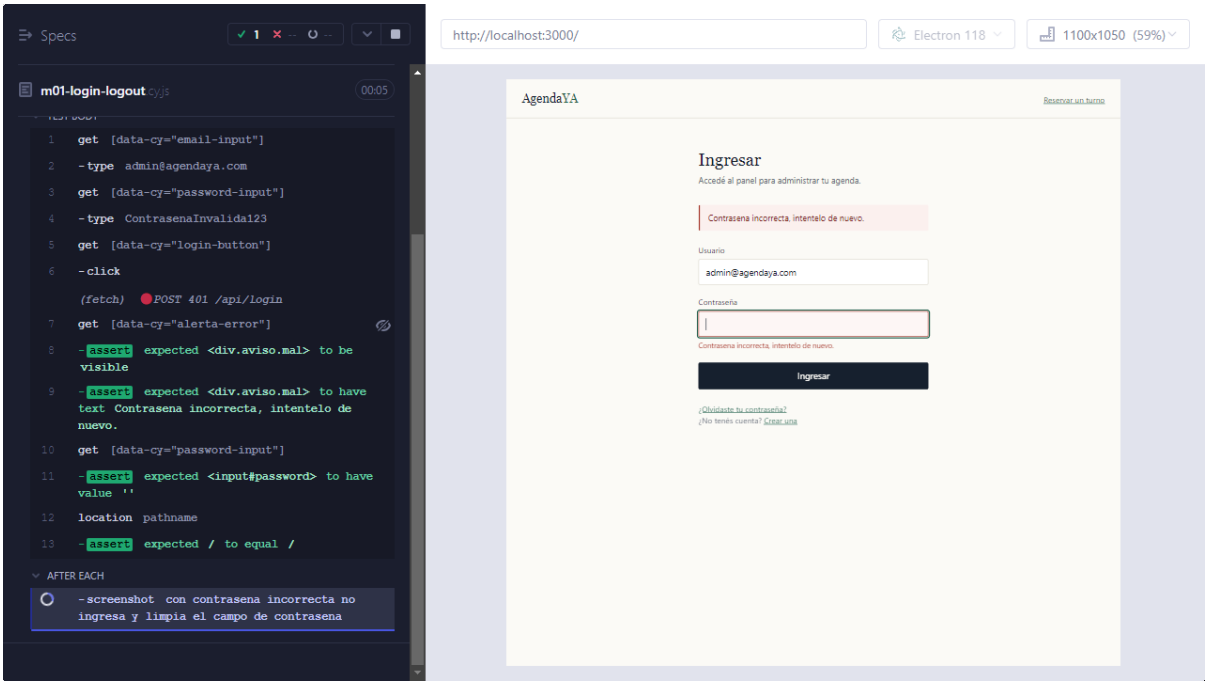


Figura 8 – m01-login-logout (Santino): contraseña incorrecta.

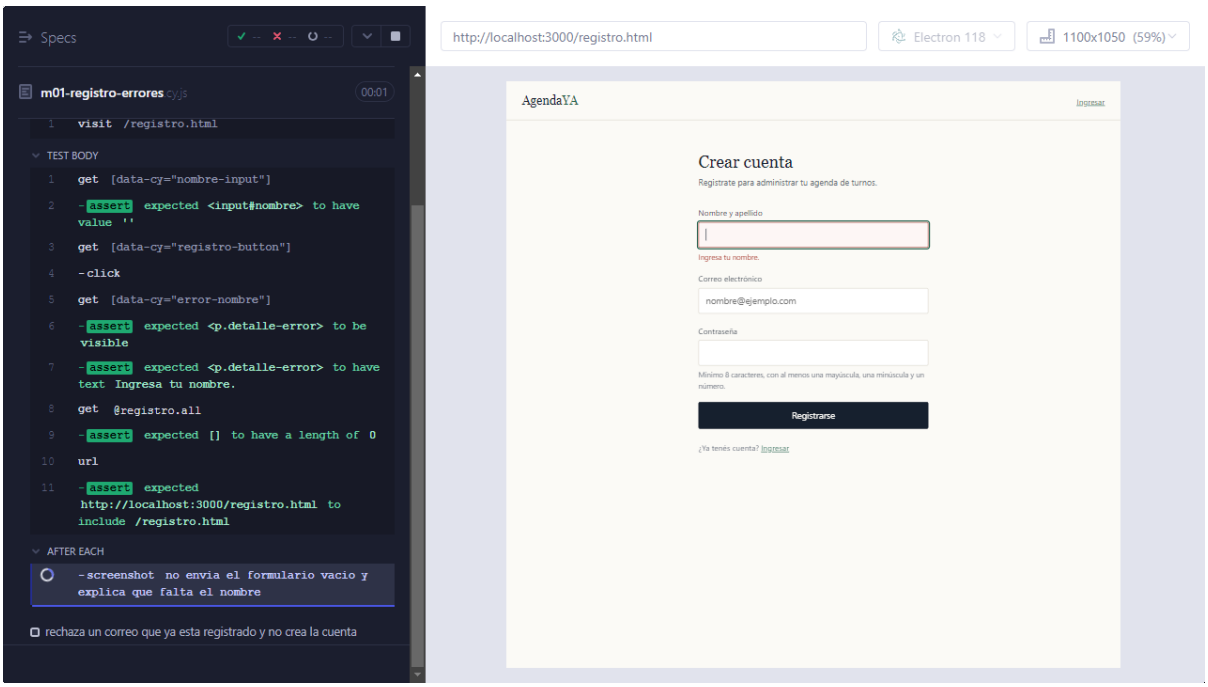


Figura 9 – m01-registro-errores (Máximo): formulario vacío.

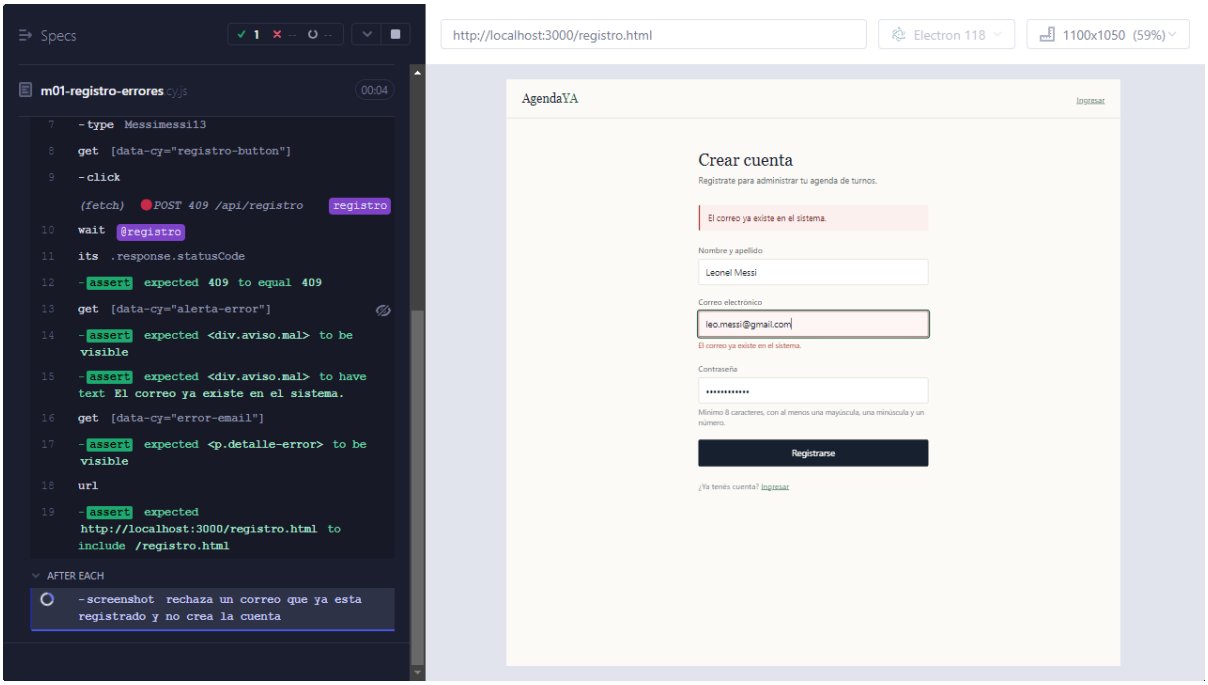


Figura 10 – m01-registro-errores (Máximo): correo ya registrado.

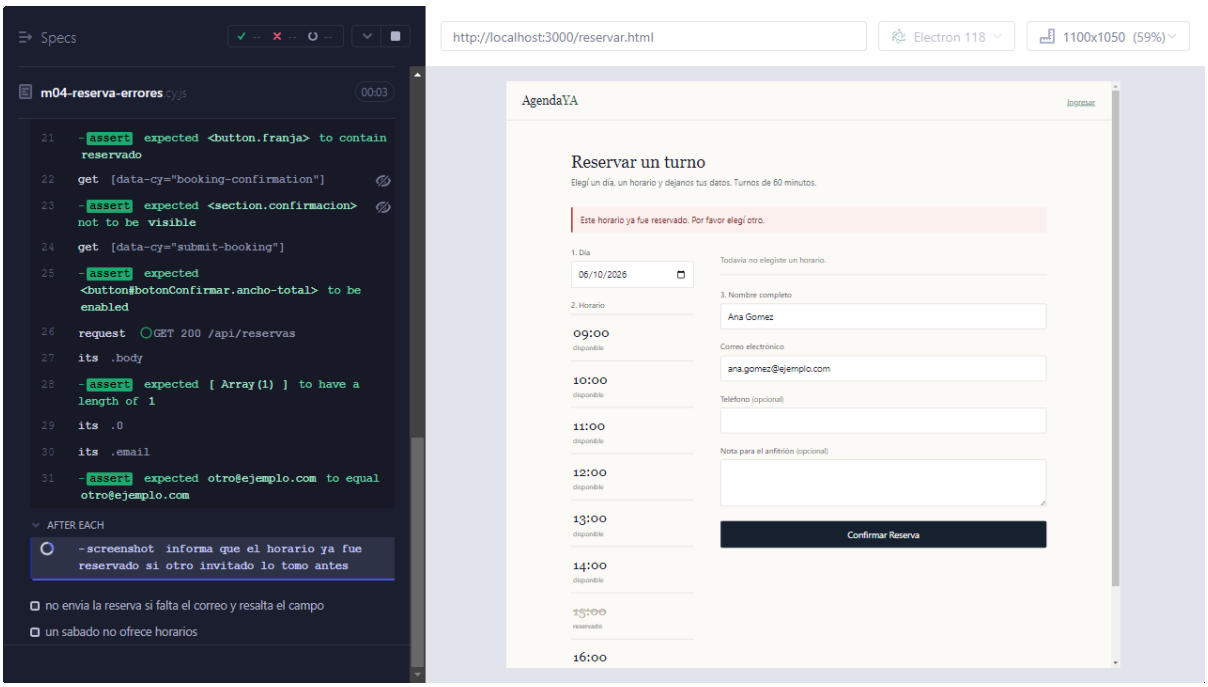


Figura 11 – m04-reserva-errores (Máximo): horario tomado por otro invitado.

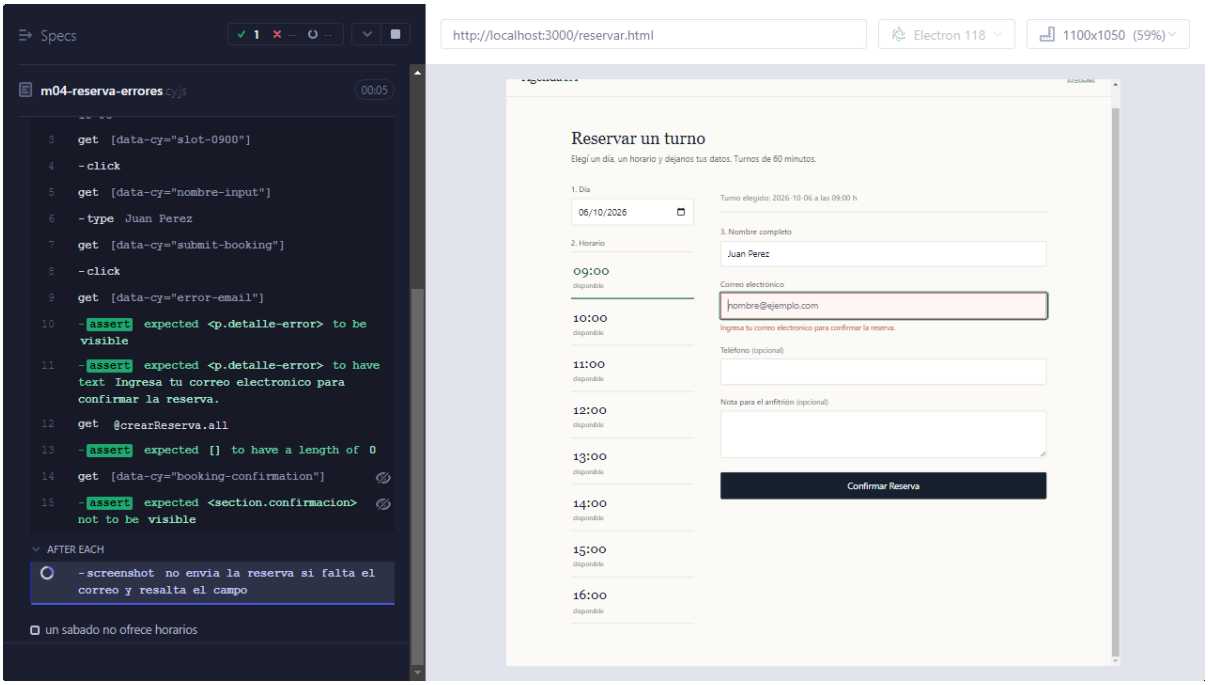


Figura 12 – m04-reserva-errores (Máximo): falta el correo.

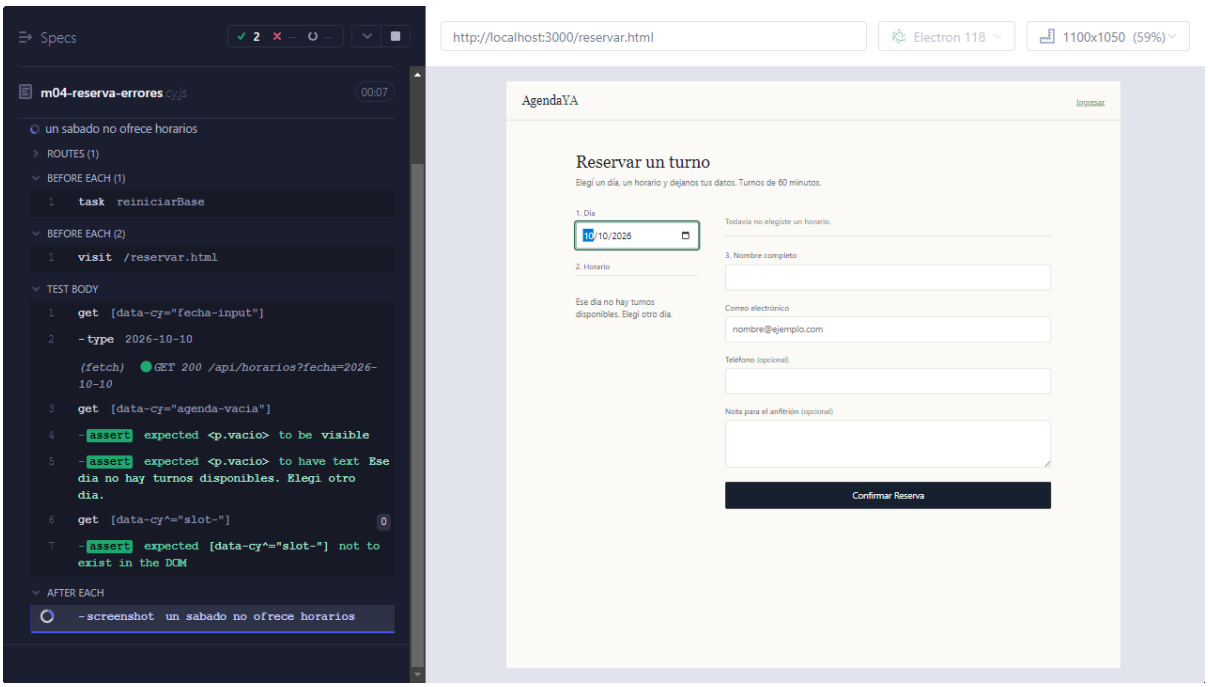


Figura 13 – m04-reserva-errores (Máximo): un sábado no hay turnos.

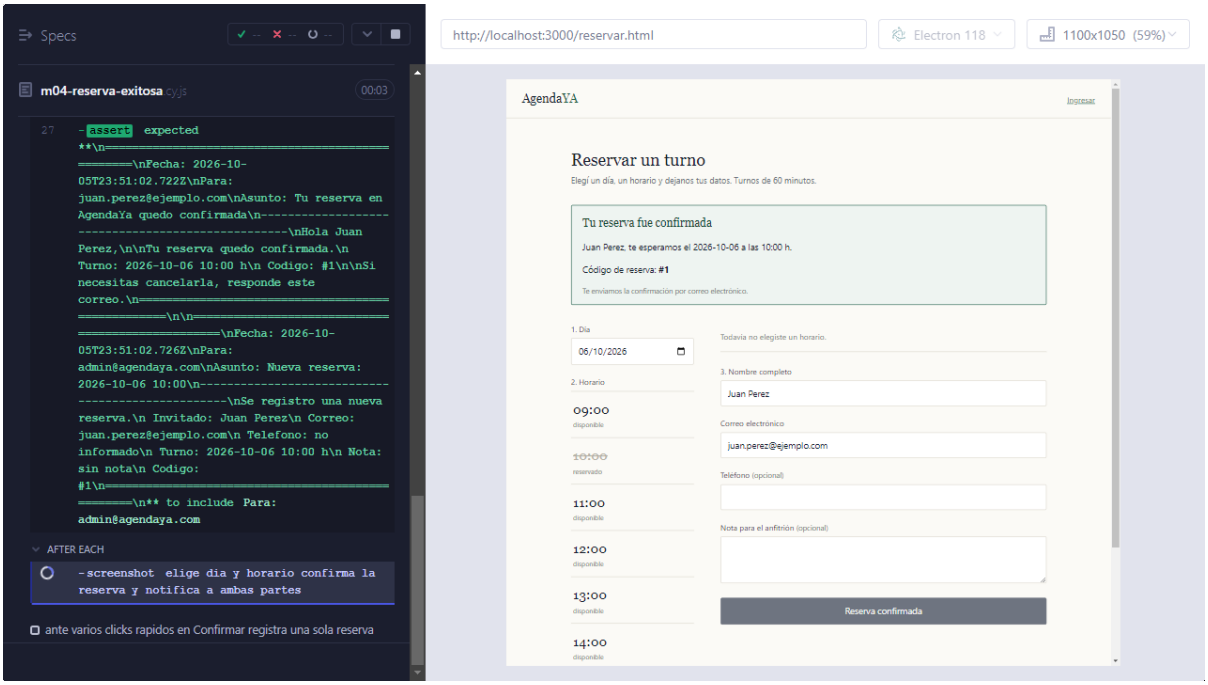


Figura 14 – m04-reserva-exitosa (Matías): reserva confirmada y correos enviados.

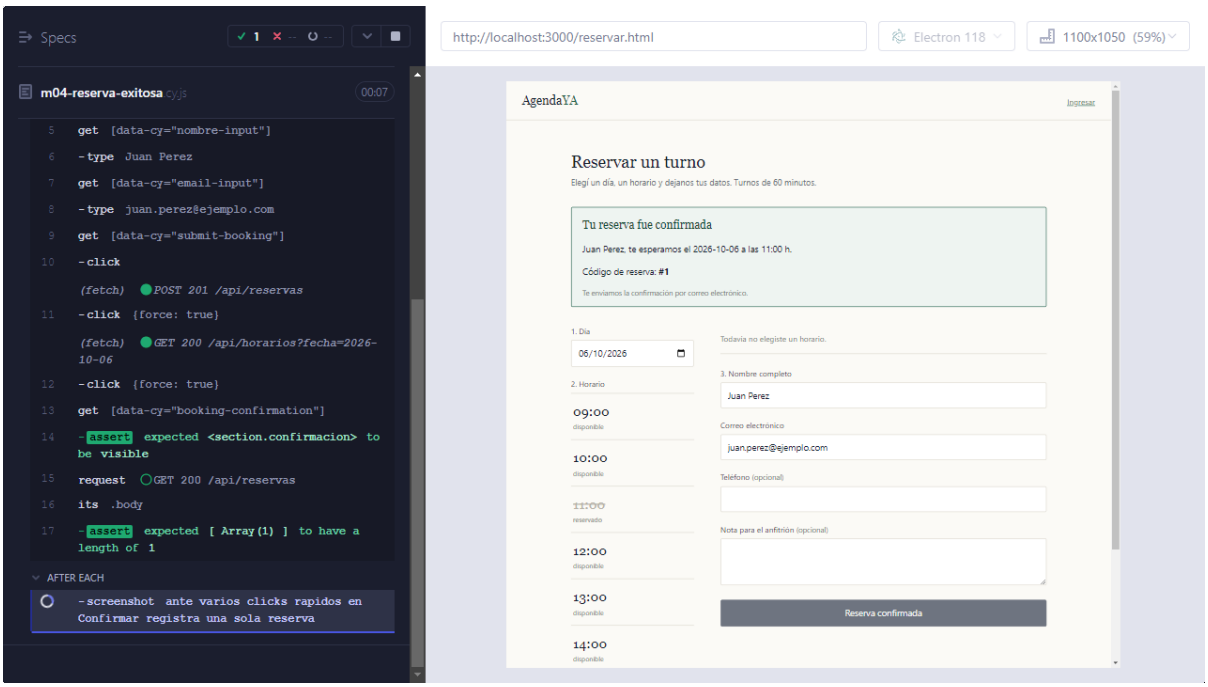


Figura 15 – m04-reserva-exitosa (Matías): varios clicks generan una sola reserva.

Análisis de resultados

En la ejecución final pasaron los 10 tests.

En la primera ejecución fallaron los 5 tests de M04, todos con el mismo error:

```
AssertionError: Timed out retrying after 4000ms: Expected to find element:
`[data-cy="slot-1000"]`, but never found it.
    at Context.eval (webpack://agendaya-tp6-grupo4/./cypress/e2e/m04-reserva-
exitosa.cy.js:44:7)
```

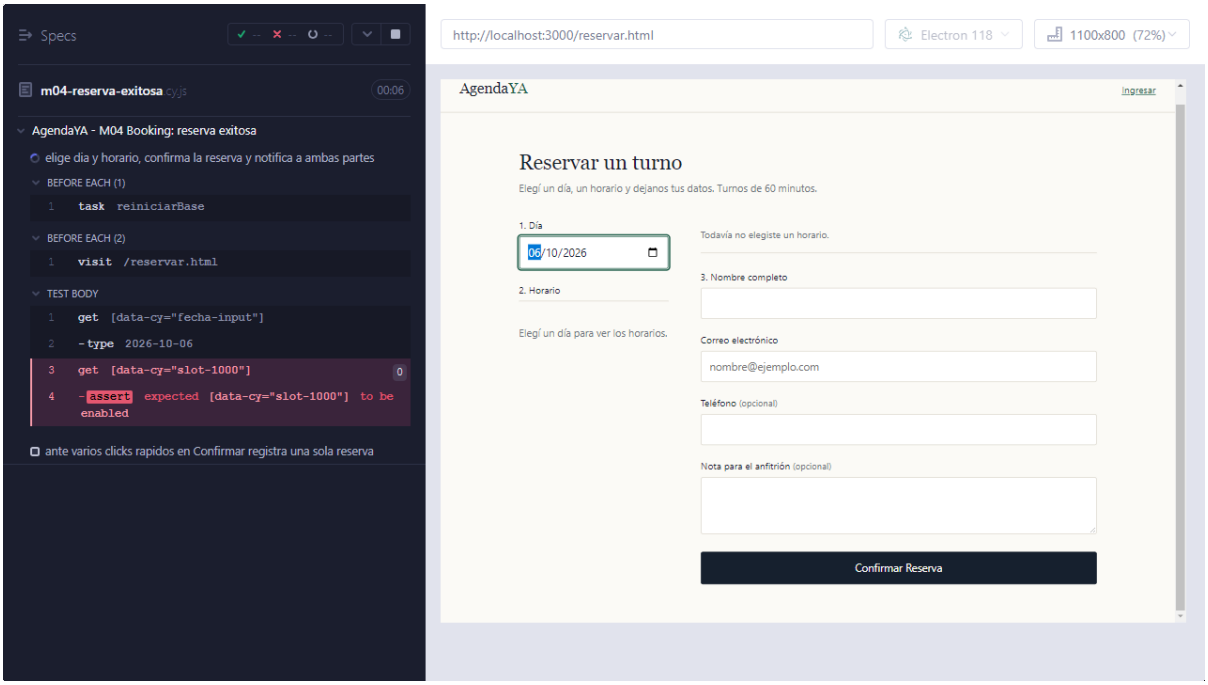


Figura 16 – Primera ejecución: la fecha está cargada pero no aparecen los horarios.

En la captura se ve la fecha cargada y la agenda vacía, con el texto "Elegí un día para ver los horarios". La pantalla cargaba los horarios con el evento change del campo de fecha, y cy.type() dispara el evento input. Decidimos corregir el frontend y no el test, porque el problema no es solo de Cypress: según el navegador y cómo se cargue la fecha (con el teclado o con el autocompletado), se dispara un evento o el otro. Ahora la pantalla escucha los dos. Con ese cambio pasaron los 10 tests.

En el test de varios clicks no alcanza con ver que el botón se deshabilita: se consulta /api/reservas para comprobar que se guardó una sola reserva. Así se cubren las dos barreras contra la doble reserva del TP5, la del navegador y la del servidor.

Tarea C: Tests unitarios con IA

Organización

Usamos Jest. Cada integrante tiene sus archivos en tests/unit, con al menos dos funciones y casos normales, de borde e inválidos. Las funciones que dependen de la hora reciben la fecha como parámetro y los tests usan fechas fijas.

Integrante	Archivo	Funciones	Tests
Barchiesi Santino	m01-email.test.js	normalizarEmail, esEmailValido, validarLogin	8
Barchiesi Santino	m01-seguridad.test.js	hashearPassword, verificarPassword, crearTokenRecuperacion, tokenVigente	8
Estudillo Máximo	m01-password-registro.test.js	evaluarPassword, validarRegistro	8
Estudillo Máximo	m04-validacion-conflictos.test.js	validarDatosReserva, detectarConflicto, evaluarReserva	10
Navarro Matías	m04-slots-antelacion.test.js	calcularSlots, cumpleAntelacion, esDiaHabil, agendaDelDia	11

Uso de IA

Usamos Claude Code, una herramienta de IA que trabaja desde la terminal sobre el repositorio: lee los archivos, escribe código y corre comandos como npm, jest y cypress.

Prompt utilizado. Le pasamos el repositorio del TP5, el enunciado del TP6 y el caso de estudio con esta consigna:

> "Necesito que me ayudes a realizar el trabajo práctico N°6, podés usar como base lo que construimos en el TP5. Primero entendé el problema, leé todo el contexto y pensá como un software engineer."

Con lo que pide la sección 7 del enunciado, la herramienta propuso qué funciones testear y en qué archivos. Antes de escribir los tests tuvo que separar la lógica del TP5 en funciones propias, porque estaba mezclada con el servidor y las pantallas. En cada integrante agregamos también un prompt más acotado, que sirve para generar ese bloque con cualquier herramienta.

Output generado. Es el código de los commits "Extraer la logica de negocio..." y "Tests unitarios con Jest...". En esa primera versión había 43 tests. Abajo está el código final de cada integrante.

Problemas que encontramos al revisar lo generado:

Qué pasó	Qué hicimos
Los 43 tests unitarios pasaron en la primera ejecución.	Un test que nunca falló no demuestra mucho. Metimos a mano dos errores en reservas.js: la anticipación con > en vez de >= (falló 1 test) y turnos que se pasan del horario de fin (fallaron 2). Después volvimos el código atrás.
En un test de login escribió "Hola, Administrador AgendaYA".replace("YA", "Ya") para que coincidiera con el nombre cargado.	Lo cambiamos por el texto esperado tal cual. Un test que acomoda el valor esperado no deja ver qué está verificando.
Un comentario // Assert decía que se verificaba el texto "Procesando reserva...", pero no había ninguna verificación de eso.	Corregimos el comentario para que diga lo que realmente se verifica.
La pantalla de reserva que generó escuchaba solo el evento change de la fecha.	Lo detectó Cypress y lo corregimos en el frontend (ver Tarea B).

Lo que no pudo resolver sola. Hubo decisiones que los requerimientos no definían y que la herramienta tuvo que tomar: qué reglas debe cumplir una contraseña segura (adoptamos 8 caracteres o más, con mayúscula, minúscula y número), si con exactamente 2 horas de anticipación se puede reservar (sí), si el enlace de recuperación sirve en el momento exacto en que vence (no) y qué disponibilidad usar, porque la define otro módulo (M02).

Tests y uso de IA por integrante

Barchiesi Santino

Archivo: tests/unit/m01-email.test.js

```
/**
 * M01 - Validacion de email
 * Responsable: Barchiesi Santino
 * Funciones: normalizarEmail, esEmailValido, validarLogin
 */
const { normalizarEmail, esEmailValido, validarLogin } =
require("../../src/logica/validaciones");

describe("normalizarEmail", () => {
  it("quita espacios y pasa a minusculas (caso normal)", () => {
    expect(normalizarEmail("  Admin@AgendaYA.com ")).toBe("admin@agendaya.com");
  });

  it("devuelve texto vacio para null o undefined (caso borde)", () => {
    expect(normalizarEmail(null)).toBe("");
```

```
    expect(normalizarEmail(undefined)).toBe("");
  });
});

describe("esEmailValido", () => {
  it("acepta un email con usuario, dominio y extension (caso normal)", () => {
    expect(esEmailValido("leo.messi@gmail.com")).toBe(true);
  });

  it("rechaza emails sin @, sin dominio o sin extension (casos invalidos)", () => {
    expect(esEmailValido("leo.messi.gmail.com")).toBe(false);
    expect(esEmailValido("leo@")).toBe(false);
    expect(esEmailValido("leo@gmail")).toBe(false);
  });

  it("rechaza espacios internos y extensiones de una sola letra (casos borde)", () => {
    expect(esEmailValido("leo messi@gmail.com")).toBe(false);
    expect(esEmailValido("leo@gmail.c")).toBe(false);
  });

  it("rechaza valores que no son texto (caso invalido)", () => {
    expect(esEmailValido(undefined)).toBe(false);
    expect(esEmailValido(12345)).toBe(false);
  });
});

describe("validarLogin", () => {
  it("con usuario y contrasena completos no devuelve error (caso normal)", () => {
    expect(validarLogin({ email: "admin@agendaya.com", password: "Contrasena123!"
})).toBeNull();
  });

  it("con el usuario vacio marca el campo email (caso invalido)", () => {
    expect(validarLogin({ email: "   ", password: "Contrasena123!" })).toEqual({
      campo: "email",
      mensaje: "Ingresa tu usuario.",
    });
  });
});
```

Archivo: tests/unit/m01-seguridad.test.js

```
/**
 * M01 - Contrasenas con hash y enlace de recuperacion
 * Responsable: Barchiesi Santino
 * Funciones: hashearPassword, verificarPassword, crearTokenRecuperacion, tokenVigente
 */
const {
  hashearPassword,
  verificarPassword,
  crearTokenRecuperacion,
  tokenVigente,
} = require("../../server/seguridad");

describe("hashearPassword / verificarPassword", () => {
  it("el hash no contiene la contrasena y la verificacion la reconoce (caso normal)", () => {
    const hash = hashearPassword("Contrasena123!");
    expect(hash).not.toContain("Contrasena123!");
    expect(verificarPassword("Contrasena123!", hash)).toBe(true);
  });

  it("rechaza una contrasena distinta, aunque difiera en una sola letra (caso invalido)", ()
=> {
    const hash = hashearPassword("Contrasena123!");
```

```
    expect(verificarPassword("contrasena123!", hash)).toBe(false);
  });

  it("la misma contrasena genera hashes distintos por la sal aleatoria (caso borde)", () => {
    expect(hashearPassword("Messimessi12")).not.toBe(hashearPassword("Messimessi12"));
  });

  it("un hash guardado con formato invalido devuelve false en lugar de romper", () => {
    expect(verificarPassword("Contrasena123!", "no-es-un-hash")).toBe(false);
    expect(verificarPassword("Contrasena123!", "sal:abc")).toBe(false);
    expect(verificarPassword("Contrasena123!", undefined)).toBe(false);
  });

  it("no permite hashear una contrasena vacia", () => {
    expect(() => hashearPassword("")).toThrow("no puede estar vacia");
  });
});

describe("crearTokenRecuperacion / tokenVigente", () => {
  const ahora = new Date("2026-10-05T12:00:00.000Z");

  it("un enlace recien creado vence a los 30 minutos y esta vigente (caso normal)", () => {
    const registro = crearTokenRecuperacion("hola.comova@agendaya.com", ahora);
    expect(registro.expiraEn).toBe("2026-10-05T12:30:00.000Z");
    expect(registro.token).toMatch(/^[0-9a-f]{48}$/);
    expect(tokenVigente(registro, ahora)).toBe(true);
  });

  it("vale un milisegundo antes del vencimiento y ya no vale en el instante exacto (caso
borde)", () => {
    const registro = { expiraEn: "2026-10-05T12:30:00.000Z" };
    expect(tokenVigente(registro, new Date("2026-10-05T12:29:59.999Z"))).toBe(true);
    expect(tokenVigente(registro, new Date("2026-10-05T12:30:00.000Z"))).toBe(false);
  });

  it("un registro inexistente o con fecha ilegible no es vigente (caso invalido)", () => {
    expect(tokenVigente(null, ahora)).toBe(false);
    expect(tokenVigente({ expiraEn: "manana" }, ahora)).toBe(false);
  });
});
```

Prompt:

> "Tengo un módulo JavaScript con normalizarEmail(valor), que devuelve el email sin espacios y en minúsculas (o "" si es null/undefined), esEmailValido(email), que valida la forma usuario@dominio.ext con extensión de 2 letras o más, y validarLogin({email, password}), que devuelve null o el primer error {campo, mensaje}. También hashearPassword/verificarPassword con scrypt y sal aleatoria, y tokenVigente(registro, ahora) para un enlace que vence a los 30 minutos. [código]. Generá tests con Jest con casos normales, de borde e inválidos. Usá fechas fijas."

Output generado: el archivo tal como quedó en el commit "Tests unitarios con Jest...".

Modificaciones: No se cambió el código generado. Solo se cambió el responsable de m01-seguridad.test.js cuando el equipo quedó en tres integrantes.

Evaluación crítica:

- Los tests son correctos y cubren los tres tipos de caso en cada función. Los del enlace de recuperación usan fechas fijas y prueban el borde exacto: un milisegundo

antes de que venza y el momento justo del vencimiento, que es donde se suele confundir < con <=.

- El caso del hash guardado con formato inválido sirvió para encontrar un error del TP5: en ese caso la función tiraba una excepción en vez de rechazar el ingreso.
- Lo que falta: validarLogin tiene solo dos casos y no prueba la contraseña vacía. Además, esEmailValido usa una expresión regular simple, así que los tests verifican esa regla y no si un email es válido según el estándar (por ejemplo, dominios con acentos). Los tests de hash son los más lentos porque usan scrypt de verdad.

Estudillo Máximo

Archivo: tests/unit/m01-password-registro.test.js

```
/**
 * M01 - Contrasena segura y validacion del registro
 * Responsable: Estudillo Maximo
 * Funciones: evaluarPassword, validarRegistro
 */
const { evaluarPassword, validarRegistro } = require("../../src/logica/validaciones");

describe("evaluarPassword", () => {
  it("acepta una contrasena con 8+ caracteres, mayuscula, minuscula y numero (caso normal)",
() => {
    expect(evaluarPassword("Messimessi12")).toEqual({ valida: true, errores: [] });
  });

  it("acepta exactamente 8 caracteres y rechaza 7 (caso borde del largo minimo)", () => {
    expect(evaluarPassword("Abcdef12").valida).toBe(true);
    expect(evaluarPassword("Abcde12")).toEqual({
      valida: false,
      errores: ["Debe tener al menos 8 caracteres."],
    });
  });

  it("informa cada regla incumplida por separado (caso invalido)", () => {
    const resultado = evaluarPassword("messimessi");
    expect(resultado.valida).toBe(false);
    expect(resultado.errores).toEqual([
      "Debe incluir al menos una mayuscula.",
      "Debe incluir al menos un numero.",
    ]);
  });

  it("trata una contrasena vacia o que no es texto como invalida sin lanzar error", () => {
    expect(evaluarPassword("").valida).toBe(false);
    expect(evaluarPassword(undefined).errores).toHaveLength(4);
  });
});

describe("validarRegistro", () => {
  const valido = { nombre: "Leonel Messi", email: "leo.messi@gmail.com", password:
"Messimessi12" };

  it("con todos los datos validos no devuelve error (caso normal)", () => {
    expect(validarRegistro(valido)).toBeNull();
  });

  it("un nombre con solo espacios cuenta como vacio (caso borde)", () => {
    expect(validarRegistro({ ...valido, nombre: "   " })).toEqual({
      campo: "nombre",
      mensaje: "Ingresa tu nombre.",
```

```
    });
  });

  it("con la contrasena debil marca el campo password e incluye el motivo (caso invalido)", ()
=> {
    const error = validarRegistro({ ...valido, password: "messi" });
    expect(error.campo).toBe("password");
    expect(error.mensaje).toContain("La contrasena no es segura.");
    expect(error.mensaje).toContain("al menos 8 caracteres");
  });

  it("informa primero el campo que aparece antes en el formulario", () => {
    expect(validarRegistro({ nombre: "", email: "mal", password: "" }).campo).toBe("nombre");
    expect(validarRegistro({ nombre: "Leo", email: "mal", password: ""
}).campo).toBe("email");
  });
});
```

Archivo: tests/unit/m04-validacion-conflictos.test.js

```
/**
 * M04 - Datos obligatorios de la reserva y conflictos de turno
 * Responsable: Estudillo Maximo
 * Funciones: validarDatosReserva, detectarConflicto, evaluarReserva
 */
const {
  validarDatosReserva,
  detectarConflicto,
  evaluarReserva,
  MENSAJES,
} = require("../../src/logica/reservas");

const solicitud = {
  fecha: "2026-10-13",
  hora: "10:00",
  nombre: "Juan Perez",
  email: "juan.perez@ejemplo.com",
};

describe("validarDatosReserva", () => {
  it("con fecha, hora, nombre y correo validos no devuelve error (caso normal)", () => {
    expect(validarDatosReserva(solicitud)).toBeNull();
  });

  it("sin correo marca el campo email con el mensaje de CP-008 (caso invalido)", () => {
    expect(validarDatosReserva({ ...solicitud, email: "" })).toEqual({
      campo: "email",
      mensaje: "Ingresa tu correo electronico para confirmar la reserva.",
    });
  });

  it("distingue un correo vacio de uno mal escrito (caso borde)", () => {
    expect(validarDatosReserva({ ...solicitud, email: "juan.perez" }).mensaje).toBe(
      "Ingresa un correo electronico valido."
    );
  });

  it("sin horario elegido pide elegir uno antes que los datos personales", () => {
    expect(validarDatosReserva({ ...solicitud, hora: null, nombre: "" }).campo).toBe("hora");
  });
});

describe("detectarConflicto", () => {
  const existentes = [
```

```
    { id: 1, fecha: "2026-10-13", hora: "10:00", email: "otro@ejemplo.com", estado:
"Confirmada" },
  ];

  it("un turno libre no tiene conflicto (caso normal)", () => {
    expect(detectarConflicto(existentes, { ...solicitud, hora: "11:00" })).toBeNull();
  });

  it("un turno tomado por otro invitado devuelve 'ocupado' con el mensaje del enunciado", ()
=> {
    expect(detectarConflicto(existentes, solicitud)).toEqual({
      tipo: "ocupado",
      mensaje: "Este horario ya fue reservado. Por favor elegí otro.",
    });
  });

  it("el mismo invitado reenviando su reserva es 'duplicada', aunque cambie mayusculas (CP-
010)", () => {
    const repetida = { ...solicitud, email: "  OTRO@ejemplo.com " };
    expect(detectarConflicto(existentes, repetida).tipo).toBe("duplicada");
  });

  it("una reserva cancelada libera el turno (caso borde)", () => {
    const cancelada = [{ ...existentes[0], estado: "Cancelada" }];
    expect(detectarConflicto(cancelada, solicitud)).toBeNull();
  });
});

describe("evaluarReserva (regla completa que aplica el servidor)", () => {
  const ahora = new Date(2026, 9, 12, 9, 0); // lunes 12/10/2026 09:00

  it("rechaza con 400 un turno de hoy sin la antelacion minima de 2 horas", () => {
    const rechazo = evaluarReserva({ ...solicitud, fecha: "2026-10-12", hora: "10:00" }, [],
ahora);
    expect(rechazo.codigo).toBe(400);
    expect(rechazo.mensaje).toBe("Los turnos se reservan con al menos 2 horas de
anticipacion.");
  });

  it("rechaza con 409 un turno ocupado y acepta (null) uno libre", () => {
    const ocupado = [{ fecha: "2026-10-13", hora: "10:00", email: "otro@ejemplo.com", estado:
"Confirmada" }];
    expect(evaluarReserva(solicitud, ocupado, ahora)).toMatchObject({ codigo: 409, mensaje:
MENSAJES.ocupado });
    expect(evaluarReserva(solicitud, [], ahora)).toBeNull();
  });
});
```

Prompt:

> "Tengo evaluarPassword(password), que devuelve {valida, errores} (mínimo 8 caracteres, una mayúscula, una minúscula y un número), y validarRegistro, que devuelve el primer error en el orden nombre, email y password. También validarDatosReserva y detectarConflicto(reservas, solicitud), que devuelve null, {tipo: "duplicada"} si el mismo email ya reservó ese turno u {tipo: "ocupado"} si lo reservó otra persona; las reservas canceladas no cuentan. [código]. Generá tests con Jest con caso normal, casos de borde (7 y 8 caracteres, mayúsculas en el email, reserva cancelada) y casos inválidos."

Output generado: el archivo tal como quedó en el commit "Tests unitarios con Jest...".

Modificaciones: No se cambió el código generado. Solo se cambió el responsable de m04-validacion-conflictos.test.js cuando el equipo quedó en tres integrantes.

Evaluación crítica:

- Los tests prueban el borde exacto del largo de la contraseña (7 caracteres no, 8 sí) y comparan la lista completa de errores, no solo si es válida o no. En los conflictos distinguen entre la reserva duplicada (el mismo invitado que envía dos veces, CP-010) y el turno ocupado por otro, y usan el mensaje tal como está en el enunciado.
- Las reglas de la contraseña las eligió la herramienta, porque el requerimiento no las especificaba. Los tests confirman que el código cumple esa regla, pero no que la regla sea la que corresponde.
- Los tests de evaluarReserva usan la disponibilidad que viene por defecto (lunes a viernes, de 9 a 17 h). Si M02 la cambia, van a fallar aunque la lógica esté bien. Hubiera sido mejor pasarles una configuración propia, como en los tests de Matías.

Navarro Matías

Archivo: tests/unit/m04-slots-antelacion.test.js

```
/**
 * M04 - Calculo de turnos y antelacion minima
 * Responsable: Navarro Matias
 * Funciones: calcularSlots, cumpleAntelacion, esDiaHabil, agendaDelDia
 */
const { calcularSlots, cumpleAntelacion, esDiaHabil, agendaDelDia } =
require("../../src/logica/reservas");

describe("calcularSlots", () => {
  it("genera un turno por hora entre el inicio y el fin (caso normal)", () => {
    expect(calcularSlots("09:00", "12:00", 60)).toEqual(["09:00", "10:00", "11:00"]);
  });

  it("no ofrece un turno que terminaria despues del horario de fin (caso borde)", () => {
    expect(calcularSlots("09:00", "10:30", 60)).toEqual(["09:00"]);
    expect(calcularSlots("09:00", "10:30", 45)).toEqual(["09:00", "09:45"]);
  });

  it("devuelve una lista vacia si el inicio y el fin son iguales o la duracion no entra", ()
=> {
    expect(calcularSlots("09:00", "09:00", 30)).toEqual([]);
    expect(calcularSlots("09:00", "09:30", 60)).toEqual([]);
  });

  it("lanza un error ante una duracion o una hora invalidas (caso invalido)", () => {
    expect(() => calcularSlots("09:00", "12:00", 0)).toThrow("entero positivo");
    expect(() => calcularSlots("09:00", "12:00", -30)).toThrow("entero positivo");
    expect(() => calcularSlots("9am", "12:00", 60)).toThrow("Hora invalida");
  });
});

describe("cumpleAntelacion", () => {
  const ahora = new Date(2026, 9, 12, 8, 0); // lunes 12/10/2026 08:00 (hora local)

  it("acepta un turno con mas de 2 horas de anticipacion (caso normal)", () => {
    expect(cumpleAntelacion(new Date(2026, 9, 12, 11, 0), ahora, 2)).toBe(true);
  });

  it("acepta exactamente 2 horas y rechaza 1 minuto menos (caso borde)", () => {
    expect(cumpleAntelacion(new Date(2026, 9, 12, 10, 0), ahora, 2)).toBe(true);
```

```
    expect(cumpleAntelacion(new Date(2026, 9, 12, 9, 59), ahora, 2)).toBe(false);
  });

  it("rechaza turnos pasados o en el instante actual, aun sin antelacion exigida (caso
invalido)", () => {
    expect(cumpleAntelacion(new Date(2026, 9, 11, 10, 0), ahora, 2)).toBe(false);
    expect(cumpleAntelacion(ahora, ahora, 0)).toBe(false);
  });
});

describe("esDiaHabil", () => {
  const config = { diasHabiles: [1, 2, 3, 4, 5], diasBloqueados: ["2026-10-12"] };

  it("un dia de semana sin bloqueo es habil y un sabado no", () => {
    expect(esDiaHabil("2026-10-13", config)).toBe(true); // martes
    expect(esDiaHabil("2026-10-17", config)).toBe(false); // sabado
  });

  it("un dia bloqueado por el administrador o una fecha imposible no son habiles", () => {
    expect(esDiaHabil("2026-10-12", config)).toBe(false); // lunes bloqueado
    expect(esDiaHabil("2026-02-30", config)).toBe(false); // no existe
  });
});

// Agregado tras la revision critica: el reporte de cobertura mostraba que
// agendaDelDia (la funcion que combina todas las reglas anteriores) no tenia
// tests unitarios y solo quedaba cubierta por los E2E.
describe("agendaDelDia", () => {
  const config = {
    diasHabiles: [1, 2, 3, 4, 5],
    inicio: "09:00",
    fin: "13:00",
    duracionMin: 60,
    antelacionMinHoras: 2,
    diasBloqueados: [],
  };
  const ahora = new Date(2026, 9, 12, 9, 30); // lunes 12/10/2026 09:30

  it("marca cada turno de hoy con el motivo por el que no se puede tomar", () => {
    const reservas = [{ fecha: "2026-10-12", hora: "12:00", email: "x@y.com", estado:
"Confirmada" }];
    expect(agendaDelDia("2026-10-12", reservas, ahora, config)).toEqual({
      fecha: "2026-10-12",
      habil: true,
      slots: [
        { hora: "09:00", disponible: false, motivo: "pasado" },
        { hora: "10:00", disponible: false, motivo: "antelacion" },
        { hora: "11:00", disponible: false, motivo: "antelacion" },
        { hora: "12:00", disponible: false, motivo: "ocupado" },
      ],
    });
  });

  it("un dia no habil no ofrece turnos", () => {
    expect(agendaDelDia("2026-10-17", [], ahora, config)).toEqual({
      fecha: "2026-10-17",
      habil: false,
      slots: [],
    });
  });
});
```

Prompt:

"Tengo una función JavaScript calcularSlots(inicio, fin, duracionMin) que recibe horas "HH:MM" y una duración en minutos y devuelve un array con la hora de inicio de cada turno que entra completo en el rango; tira un error si la duración no es un entero positivo o si la hora es inválida. También cumpleAntelacion(fechaHora, ahora, horasMin). [código]. Generá al menos 5 tests con Jest que cubran el caso normal, una duración mayor al rango disponible, horarios iguales, valores inválidos y el borde exacto de 2 horas de anticipación."

Output generado: el archivo tal como quedó en el commit "Tests unitarios con Jest...".

Modificaciones: Se agregaron 2 tests de agendaDelDia. En el reporte de cobertura vimos que esa función no tenía tests unitarios. Es la que junta todas las reglas anteriores (turno pasado, sin anticipación y ocupado) y solo la probaban los tests E2E. Con esos tests, la cobertura de líneas de reservas.js subió del 81 % al 92 %.

Evaluación crítica:

- Este bloque detectó los dos errores que metimos a propósito para probar los tests: la anticipación con > en vez de >= y un turno que se pasa del horario de fin. Prueba que no se ofrezca un turno que no entra completo: entre las 09:00 y las 10:30, con turnos de 60 minutos, devuelve solo el de las 09:00.
- Las fechas se arman con new Date(año, mes, día, hora), en hora local, así que los tests dan lo mismo en cualquier zona horaria.
- No se probó el cambio de horario de verano (hoy Argentina no lo usa). La herramienta tampoco se dio cuenta sola de que faltaban tests de agendaDelDia; lo vimos en el reporte de cobertura.

Evidencia de ejecución

Ejecución con npx jest --verbose --coverage:

```
PASS tests/unit/m04-validacion-conflictos.test.js
  validarDatosReserva
    ✓ con fecha, hora, nombre y correo validos no devuelve error (caso normal) (15 ms)
    ✓ sin correo marca el campo email con el mensaje de CP-008 (caso invalido) (3 ms)
    ✓ distingue un correo vacio de uno mal escrito (caso borde) (1 ms)
    ✓ sin horario elegido pide elegir uno antes que los datos personales (1 ms)
  detectarConflicto
    ✓ un turno libre no tiene conflicto (caso normal) (1 ms)
    ✓ un turno tomado por otro invitado devuelve 'ocupado' con el mensaje del enunciado
    ✓ el mismo invitado reenviando su reserva es 'duplicada', aunque cambie mayusculas (CP-
010) (7 ms)
    ✓ una reserva cancelada libera el turno (caso borde) (1 ms)
  evaluarReserva (regla completa que aplica el servidor)
    ✓ rechaza con 400 un turno de hoy sin la antelacion minima de 2 horas (3 ms)
    ✓ rechaza con 409 un turno ocupado y acepta (null) uno libre (3 ms)

PASS tests/unit/m04-slots-antelacion.test.js
  calcularSlots
    ✓ genera un turno por hora entre el inicio y el fin (caso normal) (20 ms)
    ✓ no ofrece un turno que terminaria despues del horario de fin (caso borde) (1 ms)
    ✓ devuelve una lista vacia si el inicio y el fin son iguales o la duracion no entra (1 ms)
    ✓ lanza un error ante una duracion o una hora invalidas (caso invalido) (73 ms)
  cumpleAntelacion
    ✓ acepta un turno con mas de 2 horas de anticipacion (caso normal) (2 ms)
    ✓ acepta exactamente 2 horas y rechaza 1 minuto menos (caso borde) (1 ms)
```

```
    ✓ rechaza turnos pasados o en el instante actual, aun sin antelacion exigida (caso
invalido) (1 ms)
  esDiaHabil
    ✓ un dia de semana sin bloqueo es habil y un sabado no (1 ms)
    ✓ un dia bloqueado por el administrador o una fecha imposible no son habiles (1 ms)
  agendaDelDia
    ✓ marca cada turno de hoy con el motivo por el que no se puede tomar (2 ms)
    ✓ un dia no habil no ofrece turnos (1 ms)

PASS tests/unit/m01-password-registro.test.js
  evaluarPassword
    ✓ acepta una contrasena con 8+ caracteres, mayuscula, minuscula y numero (caso normal) (3
ms)
    ✓ acepta exactamente 8 caracteres y rechaza 7 (caso borde del largo minimo) (2 ms)
    ✓ informa cada regla incumplida por separado (caso invalido) (2 ms)
    ✓ trata una contrasena vacia o que no es texto como invalida sin lanzar error (1 ms)
  validarRegistro
    ✓ con todos los datos validos no devuelve error (caso normal) (1 ms)
    ✓ un nombre con solo espacios cuenta como vacio (caso borde) (1 ms)
    ✓ con la contrasena debil marca el campo password e incluye el motivo (caso invalido) (1
ms)
    ✓ informa primero el campo que aparece antes en el formulario (1 ms)

PASS tests/unit/m01-email.test.js
  normalizarEmail
    ✓ quita espacios y pasa a minusculas (caso normal) (1 ms)
    ✓ devuelve texto vacio para null o undefined (caso borde) (1 ms)
  esEmailValido
    ✓ acepta un email con usuario, dominio y extension (caso normal) (1 ms)
    ✓ rechaza emails sin @, sin dominio o sin extension (casos invalidos)
    ✓ rechaza espacios internos y extensiones de una sola letra (casos borde) (1 ms)
    ✓ rechaza valores que no son texto (caso invalido)
  validarLogin
    ✓ con usuario y contrasena completos no devuelve error (caso normal)
    ✓ con el usuario vacio marca el campo email (caso invalido) (1 ms)

PASS tests/unit/m01-seguridad.test.js
  hashearPassword / verificarPassword
    ✓ el hash no contiene la contrasena y la verificacion la reconoce (caso normal) (294 ms)
    ✓ rechaza una contrasena distinta, aunque difiera en una sola letra (caso invalido) (211
ms)
    ✓ la misma contrasena genera hashes distintos por la sal aleatoria (caso borde) (204 ms)
    ✓ un hash guardado con formato invalido devuelve false en lugar de romper (2 ms)
    ✓ no permite hashear una contrasena vacia (34 ms)
  crearTokenRecuperacion / tokenVigente
    ✓ un enlace recien creado vence a los 30 minutos y esta vigente (caso normal) (4 ms)
    ✓ vale un milisegundo antes del vencimiento y ya no vale en el instante exacto (caso
borde) (1 ms)
    ✓ un registro inexistente o con fecha ilegible no es vigente (caso invalido) (1 ms)

------------------|---------|----------|---------|---------|---------------------------
File              | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
------------------|---------|----------|---------|---------|---------------------------
All files         |   91.66 |       80 |   96.66 |   93.95 |
 server           |     100 |     91.3 |     100 |     100 |
  seguridad.js    |     100 |     91.3 |     100 |     100 | 37,54
 src/logica       |   90.38 |    78.03 |   96.15 |   93.02 |
  reservas.js     |   90.56 |    77.38 |   94.73 |   92.13 | 14,80,160,166,209,212,216
  validaciones.js |      90 |    79.16 |     100 |      95 | 13,81
------------------|---------|----------|---------|---------|---------------------------
Test Suites: 5 passed, 5 total
Tests:       45 passed, 45 total
Snapshots:   0 total
Time:        2.854 s, estimated 3 s
Ran all test suites.
```

Algunas de las líneas sin cubrir solo se ejecutan en el navegador (la carga del módulo con <script> y fechaIso, que calcula la fecha mínima del calendario) o son valores por defecto de parámetros opcionales. Otras son rechazos que no tienen un test unitario propio: en validarDatosReserva, la fecha vacía y el nombre vacío; en evaluarReserva, el día no hábil, la hora fuera de la agenda y el turno pasado. El día no hábil está probado en un test E2E, pero solo en la pantalla. El resto queda pendiente para la próxima entrega.

Reflexión estructurada

Sobre la trazabilidad

En general sí. Los casos de prueba del TP5, que salieron de los requerimientos, se pasaron a tests casi sin cambios. Los que tenían el mensaje exacto del sistema (CP-002, CP-004 y CP-008) se convirtieron directamente en verificaciones. Los problemas aparecieron donde el requerimiento dejaba algo sin definir. AYA-M01-RF-01 pide una contraseña "que cumpla con los requisitos de seguridad", pero no dice cuáles, y para escribir evaluarPassword tuvimos que decidir la regla. La anticipación mínima ("no se puede reservar con menos de 2 horas") no aclara si con 2 horas justas se puede, y el test de borde nos obligó a elegir. Tampoco teníamos documentada la dependencia de M04 con M02: sin la disponibilidad del administrador no se puede probar la elección de fecha, así que tuvimos que dejarla fija.

Sobre el valor de testear

Sí. Cypress encontró que los horarios no se cargaban al escribir la fecha, porque la pantalla escuchaba un solo evento. Al preparar la elección de fecha vimos que los horarios del TP5, fijos al 15/09/2026, ya habían pasado y se podían seguir reservando, cuando el enunciado pide lo contrario. Al pensar el caso del hash con formato inválido vimos que la función del TP5 tiraba una excepción en vez de rechazar el ingreso. Y al agregar los data-cy al panel encontramos que la lista de reservas se armaba con innerHTML usando lo que escribe el invitado, lo que permitía inyectar código (XSS). Ninguno de estos errores había aparecido en las pruebas manuales del TP5. Testear sirve más si se hace mientras se programa y obliga a pensar los casos de borde, no solo como control al final.

Sobre el uso de IA

Donde más nos sirvió fue en el código fuente y el frontend: armar las pantallas con sus data-cy y configurar Cypress y Jest, que es trabajo bastante mecánico. También sirvió para pensar casos de borde de los tests unitarios. Las limitaciones aparecieron en los tests: hizo uno que acomodaba el valor esperado para que pasara, dejó un comentario que no coincidía con lo que se verificaba y no se dio cuenta de que faltaban tests de una función. Tampoco podía resolver lo que los requerimientos dejaban sin definir: tomó decisiones que después tuvimos que revisar. Por último, que todos los tests pasen no quiere decir que sirvan; para comprobarlo tuvimos que meter errores a propósito.

Lecciones aprendidas

1. **Para hacer tests unitarios hay que separar la lógica de la pantalla.** En el TP5 las reglas estaban mezcladas con el servidor y el HTML, y no había nada para probar por separado. Al pasarlas a funciones aparte pudimos escribir 45 tests que corren en unos 3 segundos y prueban el mismo código que usa la pantalla.
2. **La hora actual también es un dato de entrada.** Una función que usa new Date() adentro da distinto según el día. Pasarle la fecha como parámetro hizo que los tests de anticipación y de vencimiento den siempre lo mismo. En los E2E, usar el próximo día hábil evita que los tests dejen de andar con el tiempo.
3. **Un test que falla puede estar mostrando un error real.** El fallo de Cypress con la fecha parecía un problema de la herramienta, pero mostraba que la pantalla dependía de un solo evento del navegador. Antes de cambiar el test hay que pensar si un usuario podría pasar por lo mismo.
4. **Los data-cy hacen que los tests no dependan del diseño.** Cambiamos textos, clases y la estructura de las pantallas y no se rompió ningún test, porque los selectores apuntan a atributos que están solo para eso.
5. **Lo que genera la IA hay que revisarlo como si fuera código de otro.** Armó en minutos la estructura y la mayoría de los casos, pero también un test que acomodaba el resultado y un comentario que no correspondía. Sin revisar, esos tests nos hubieran dado una seguridad falsa.
6. **Que los tests pasen no quiere decir que sirvan.** Meter errores a propósito (cambiar un >= por >) y ver que los tests fallan es una forma rápida de comprobar que verifican algo. El reporte de cobertura nos mostró una función sin tests.
7. **Un requerimiento ambiguo se nota al escribir el test.** "Requisitos de seguridad" o "menos de 2 horas" se entienden al leerlos, pero el test pide un valor concreto. Si en el TP1 hubiéramos escrito los criterios con ejemplos (7 caracteres: no; 2 horas justas: sí), no habríamos tenido que decidirlo al programar.
8. **Hay que verificar el estado del sistema, no solo la pantalla.** Los tests que más valen revisan también lo que no se ve: que se guardó una sola reserva con varios clicks, que se mandaron los dos correos o que el servidor respondió 409. Si la pantalla se ve bien pero los datos están mal, sigue siendo un error.